„Programmiersprachen“-Abschlussprojekt

Wintersemester 2026/27

Schreiben Sie eine Konsolenanwendung, die den Ticketverkauf eines Kinos simuliert. Die Benutzerin sieht jeweils die aktuelle Liste der Tickets und kann danach wiederholt Tickets kaufen, gekaufte Tickets zurückgeben oder das Programm beenden. Das Programm soll sicher mit ungültigen Eingaben umgehen. Die genaue Formulierung der Konsolenmeldungen ist frei wählbar, sofern sie nicht vorgegeben ist.

Aufgabe 1 – 15 %

Implementieren Sie eine Klasse `Ticket`, die ein Kinoticket repräsentiert. Diese Klasse soll nicht instanziierbar sein und soll die folgenden Eigenschaften haben:

- **MovieTitle** – der Titel des Films (`string`). Von außerhalb der Klasse lesbar.
- **Price** – der reguläre Preis des Tickets (`decimal`). Nur innerhalb der Klasse oder abgeleiteter Klassen sichtbar.

`MovieTitle` und `Price` werden im Konstruktor initialisiert und können danach nicht mehr verändert werden.

Außerdem hat `Ticket` die Eigenschaft **IsSold** (`bool`), die angibt, ob das Ticket bereits verkauft ist. Sie soll von außerhalb der Klasse lesbar sein, aber nur innerhalb der Klasse oder abgeleiteter Klassen verändert werden können. Der Startwert ist `false`.

Implementieren Sie außerdem eine von außerhalb der Klasse sichtbare Methode **GetFinalPrice()**, die den tatsächlich zu zahlenden Preis als `decimal` zurückgibt. In `Ticket` hat diese Methode keine Implementierung; jede abgeleitete Klasse muss sie bereitstellen.

Aufgabe 2 – 15 %

Erstellen Sie zwei Klassen, die von `Ticket` erben:

- **StandardTicket** – der Endpreis entspricht dem regulären Preis.
- **DiscountTicket** – hat die zusätzliche Eigenschaft **DiscountPercentage** (`decimal`, nur innerhalb der Klasse sichtbar, wird im Konstruktor initialisiert). Der Endpreis ist der um `DiscountPercentage` Prozent reduzierte reguläre Preis.

Beide Klassen müssen instanziiert werden können und die Basisklasse korrekt initialisieren.

Aufgabe 3 – 15 %

Fügen Sie der Klasse `Ticket` die folgenden Methoden hinzu. Beide sollen von außerhalb der Klasse sichtbar sein.

- **Purchase()** hat keine Parameter. Ist das Ticket bereits verkauft, soll eine Exception mit einer passenden Fehlermeldung ausgelöst werden; andernfalls wird das Ticket als verkauft markiert.
- **PrintTicket()** gibt den Filmtitel, den Endpreis (durch Aufruf von `GetFinalPrice()`) und die Angabe, ob das Ticket verkauft ist, in der Konsole aus.

Aufgabe 4 – 20 %

Erstellen Sie eine Klasse `BoxOffice` (die Kinokasse). Sie soll eine `Dictionary`-Eigenschaft besitzen, die Ticketnummern (z. B. 101, 102, …) mit `Ticket`-Objekten verknüpft. Das Dictionary darf nicht außerhalb der Klasse sichtbar sein und soll mit mindestens vier Tickets initialisiert werden (darunter sowohl `StandardTicket`- als auch `DiscountTicket`-Objekte). Es dürfen auch mehrere Tickets für denselben Film existieren.

Implementieren Sie außerdem die folgenden Methoden. Beide sollen von außerhalb der Klasse aufrufbar sein.

- **ShowTickets()** durchläuft alle Einträge des Dictionarys und gibt für jedes Ticket die Ticketnummer und anschließend (mittels `PrintTicket()`) die Ticketinformationen aus.
- **BuyTicket** hat einen Parameter für die Ticketnummer. Die Methode prüft, ob die Ticketnummer im Dictionary existiert (falls nicht, wird eine Fehlermeldung angezeigt) und ruft andernfalls `Purchase()` des entsprechenden Tickets auf. Alle dabei ausgelösten Exceptions werden behandelt (Fehlermeldung anzeigen). Ein erfolgreicher Kauf wird in der Konsole bestätigt, inklusive des Endpreises.

Aufgabe 5 – 15 %

a) Erstellen Sie in der Klasse `Program` eine Methode **ReadTicketNumber()** ohne Parameter mit dem Rückgabetyp `int`. Sie fordert die Benutzerin auf, eine Ticketnummer einzugeben, und liest die Eingabe. Ist die Eingabe keine gültige ganze Zahl, wird eine Fehlermeldung angezeigt und die Eingabe wiederholt, bis eine gültige Zahl eingegeben wurde. Diese wird zurückgegeben.

b) Erstellen Sie in der `Main`-Methode eine Instanz von `BoxOffice`. Wiederholen Sie dann Folgendes, bis die Benutzerin „x“ wählt:

Zuerst wird die Liste aller Tickets angezeigt. Danach wird der Benutzerin angezeigt, welche Optionen sie hat (z. B. „b: Ticket kaufen, x: Beenden“), und sie wählt, ob sie ein Ticket kaufen („b“) oder das Programm beenden („x“) möchte. Nur wenn sie „b“ wählt, wird anschließend die Ticketnummer abgefragt und `BuyTicket` aufgerufen. Bei jeder anderen Eingabe wird die Meldung „Ungültige Eingabe“ angezeigt.

Aufgabe 6 – 20 %

Tickets sollen zurückgegeben werden können, allerdings nur `StandardTicket`s. Tickets mit Rabatt können nicht zurückgegeben werden.

a) Definieren Sie ein Interface `IRefundable` mit einer Methode `decimal Refund()`. Nur die Klasse `StandardTicket` soll dieses Interface implementieren.

b) Implementieren Sie `Refund()` in `StandardTicket`: Ist das Ticket nicht verkauft, soll eine Exception mit einer passenden Fehlermeldung ausgelöst werden. Andernfalls wird das Ticket zurückerstattet und der erstattete Betrag (= Endpreis) zurückgegeben.

Hinweis: Ein zurückerstattetes Ticket gilt nicht mehr als verkauft.

c) Erstellen Sie in `BoxOffice` eine von außerhalb der Klasse aufrufbare Methode **RefundTicket** mit einem Parameter für die Ticketnummer. Sie prüft, ob die Ticketnummer existiert (falls nicht, Fehlermeldung). Mit dem Operator `is` oder `as` prüft sie, ob das Ticket zurückgegeben werden kann (falls nicht, passende Meldung). Andernfalls ruft sie `Refund()` auf und behandelt die dabei ausgelöste Exception. Eine erfolgreiche Rückgabe wird in der Konsole bestätigt, inklusive des erstatteten Betrags.

d) Erweitern Sie das Menü in `Main` um die Option „r“ (Ticket zurückgeben). Wie beim Kaufen wird nach der Auswahl die Ticketnummer abgefragt und anschließend `RefundTicket` aufgerufen.